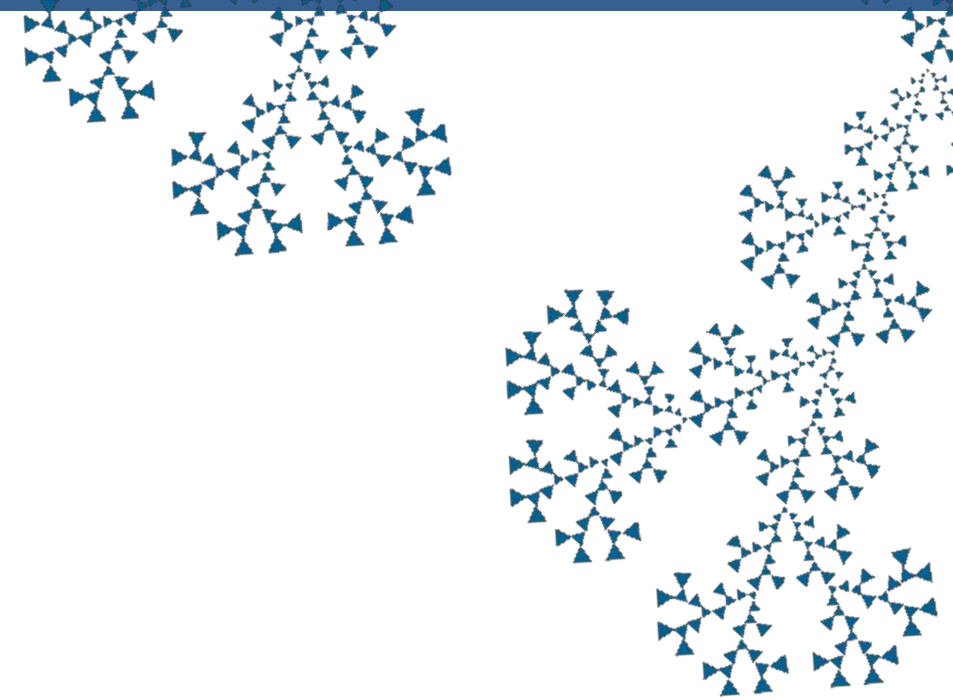




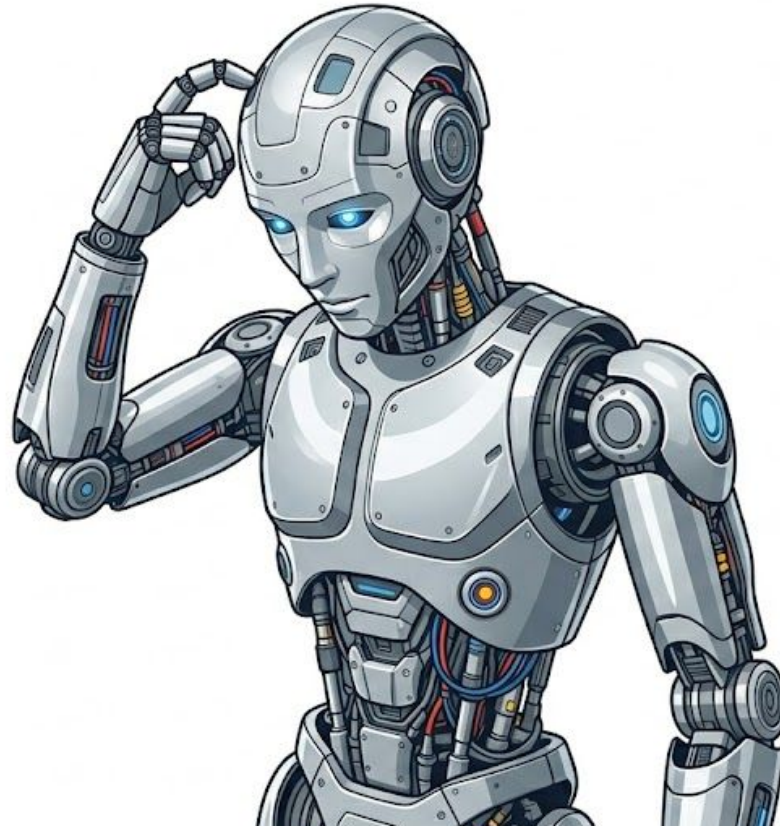
Hybrid Quantum and Classical Workload Management with Graph-based Scheduling

Vanessa Sochat and Daniel Milroy
Computer Scientists
Lawrence Livermore National Laboratory

OpenQSE Day, July 24, 2026

What might the future look like?

The center of the future is hybrid, and autonomous.



What might the future look like?

The center of the future is hybrid, and autonomous.

*How do I run classical
and quantum
workloads?*



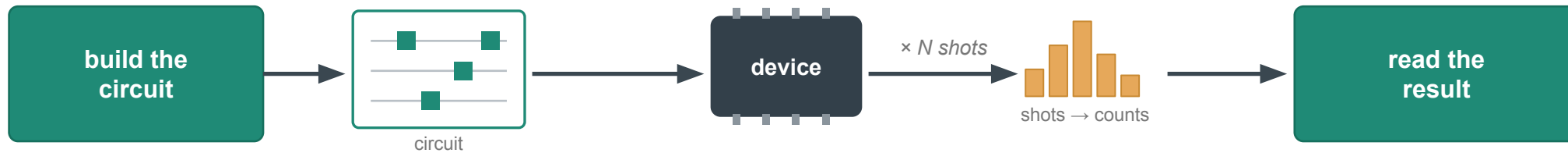
QUANTUM APPLICATIONS

Hybrid Application Designs

The designs of a workload manager and scheduler are driven by application needs

Pattern 1: Single Evaluation

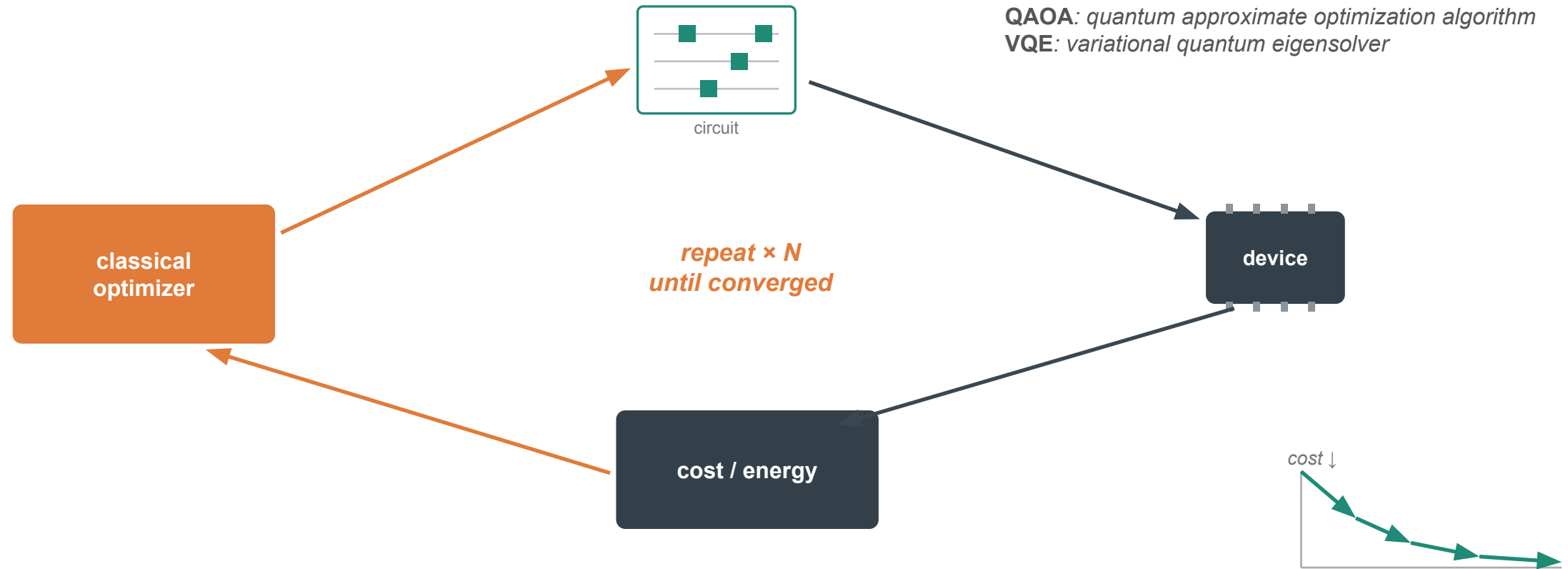
Build one circuit, run it once (many shots), read the result.



Shape: a straight pipeline — one pass, left to right. One call to the device.

Pattern 2: Variational loop (QAOA/VQE)

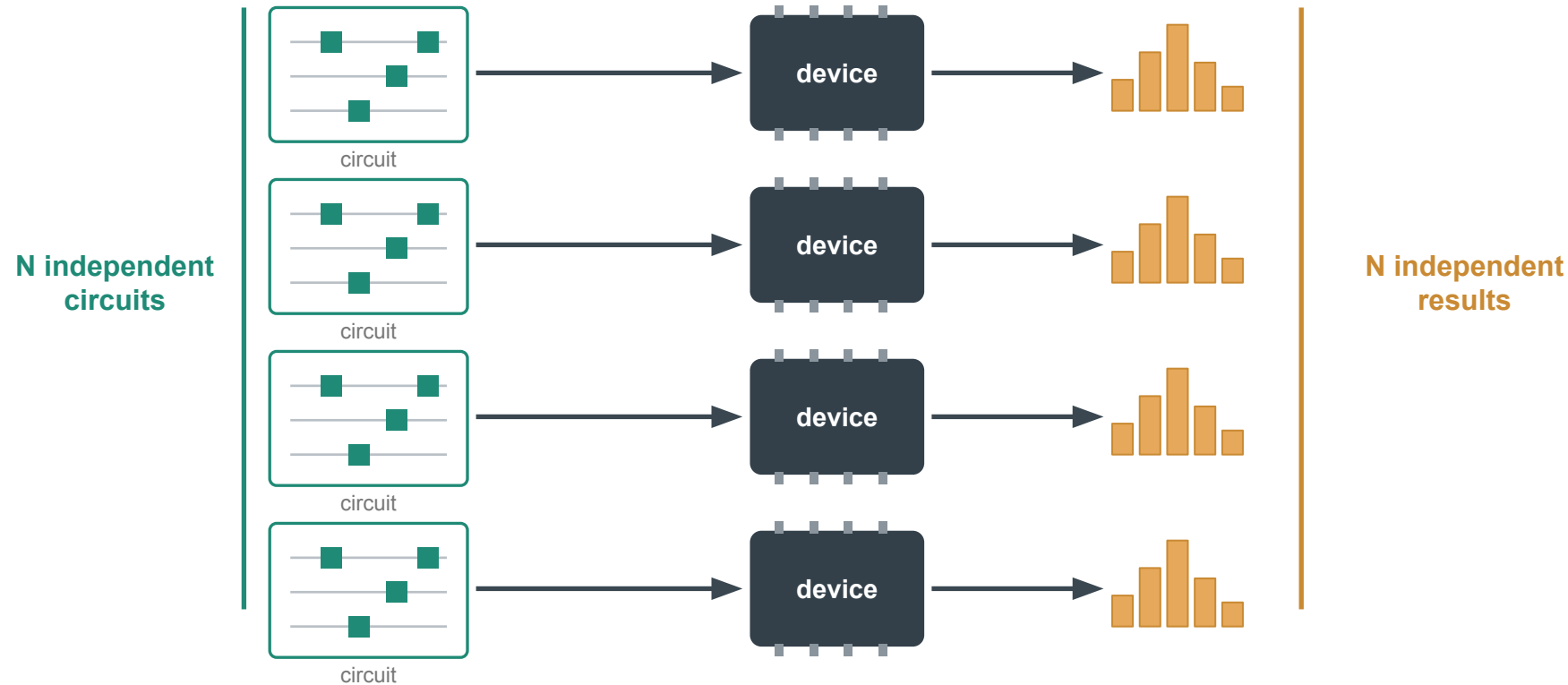
A classical optimizer proposes parameters, runs a circuit, and adjusts to convergence



Shape: a cycle — the device sits inside a feedback loop with a classical optimizer. Round-trip until the cost stops improving.

Pattern 3: Batch (embarrassingly parallel)

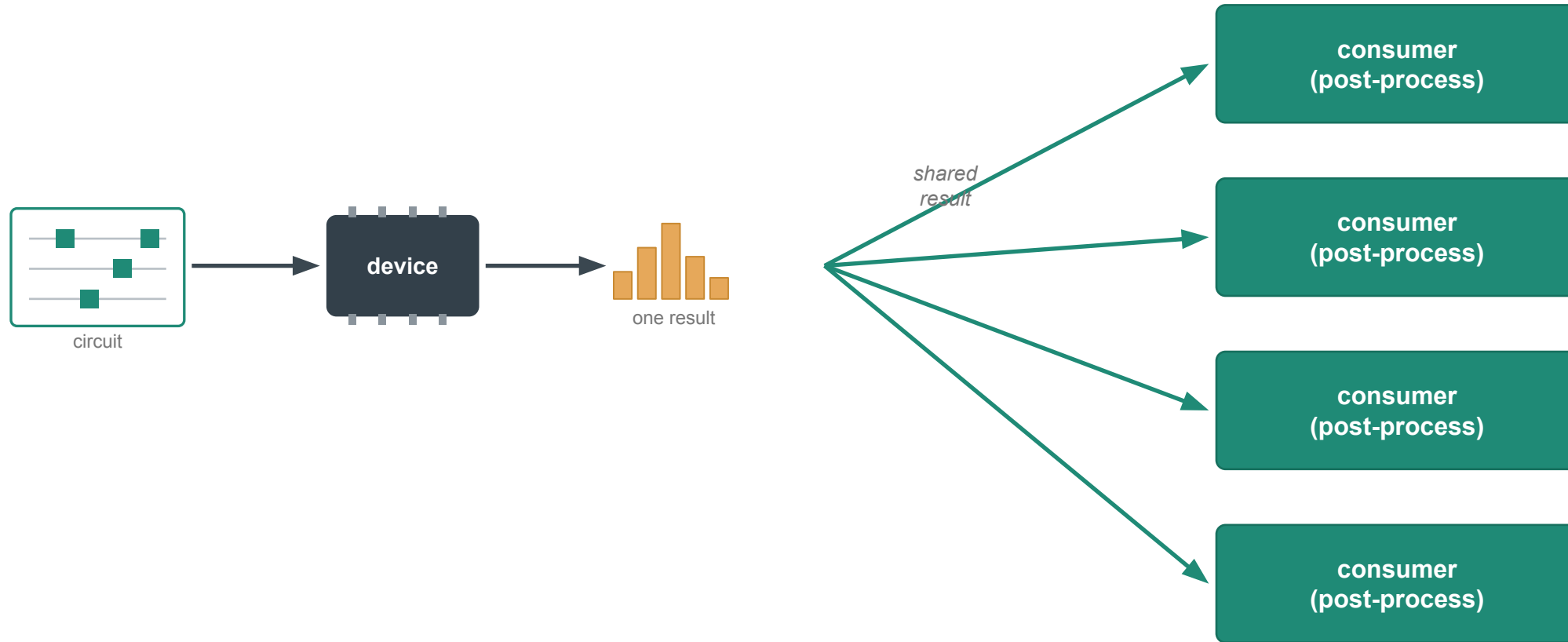
Many independent circuits at once (e.g., parameter sweep) without dependency between.



Shape: a parallel stack of pancakes, each self-contained and not connected.

Pattern 4: Fan-out (shared result / scatter-gather)

One quantum result feeds many classical consumers.



Shape: one-to-many. A single device call generates results that are fanned out to consumers.

How does application design influence scheduling?

How does application design influence scheduling?

When my application expects a resource, it needs to be there.

QUANTUM APPLICATIONS INTERSECTION WITH SCHEDULING

Scheduling Designs

How do the needs of co-scheduling resources influence our scheduler designs?

How does resource design influence scheduling?

How does resource design influence scheduling?

My ability to schedule depends on transparency and control.

How does resource design influence scheduling?

My ability to schedule depends on **transparency** and control.

TRAIN STATION



新宿駅
SHINJUKU STN.

DESTINATION

DEPARTURE TIME

REMARKS

YAMANOTE LINE
(via SHINJUKU, SHIBUYA)

10:17 ON TIME

10:18 ON TIME APPROACHING



新宿駅
SHINJUKU STN.

DESTINATION

DEPARTURE TIME

REMARKS

YAMANOTE LINE
(via SHINJUKU, SHIBUYA)

NEXT DEPARTURE:
EVENTUALLY



How does resource design influence scheduling?

My ability to schedule depends on **transparency** and control.

Without control, I need **transparency** to be on time.y

How does resource design influence scheduling?

My ability to schedule depends on transparency and **control**.

How does resource design influence scheduling?

My ability to schedule depends on transparency and **control**.



How does resource design influence scheduling?

What about intermediate control and transparency?

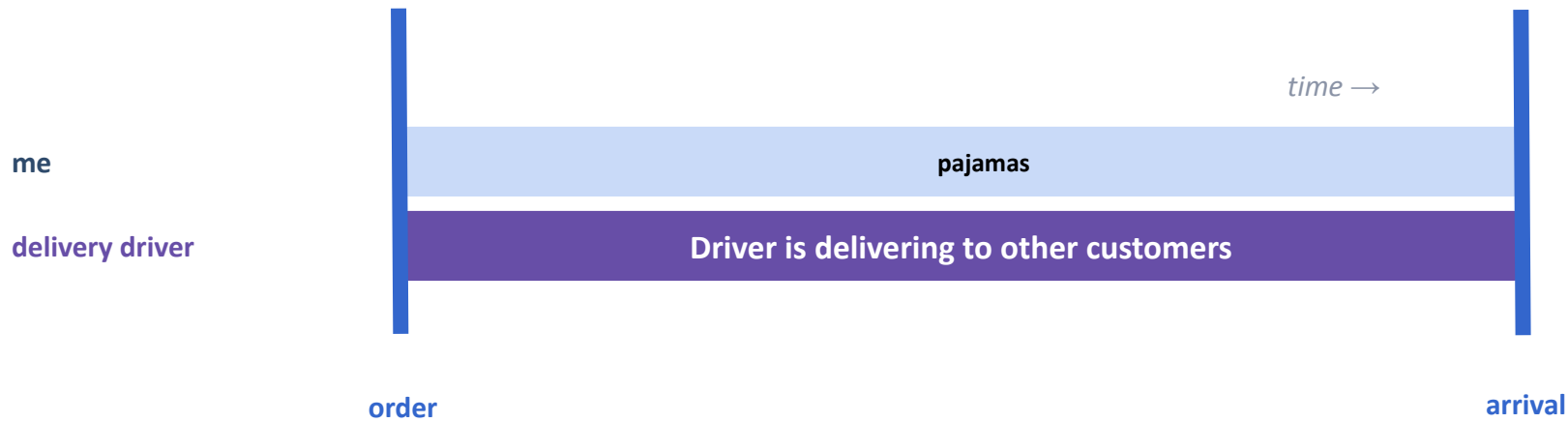






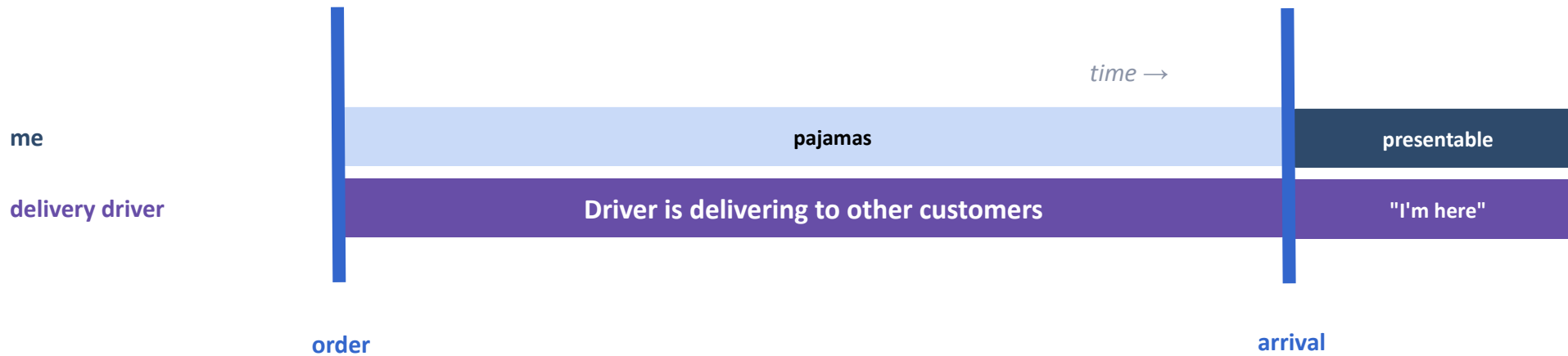
Scheduling the grocery delivery...

I need to be ready to receive the driver when he or she arrives!



Scheduling the grocery delivery...

I need to be ready to receive the driver when he or she arrives!



How does resource design influence scheduling?

What about intermediate control and transparency?

*When we don't have complete control or transparency,
guaranteed and **predictable** is a good start.*

THE NEW WORK

The two-queue problem

A remote QPU adds a second queue the scheduler cannot see or control — and classical workers idle while a quantum task waits in it.

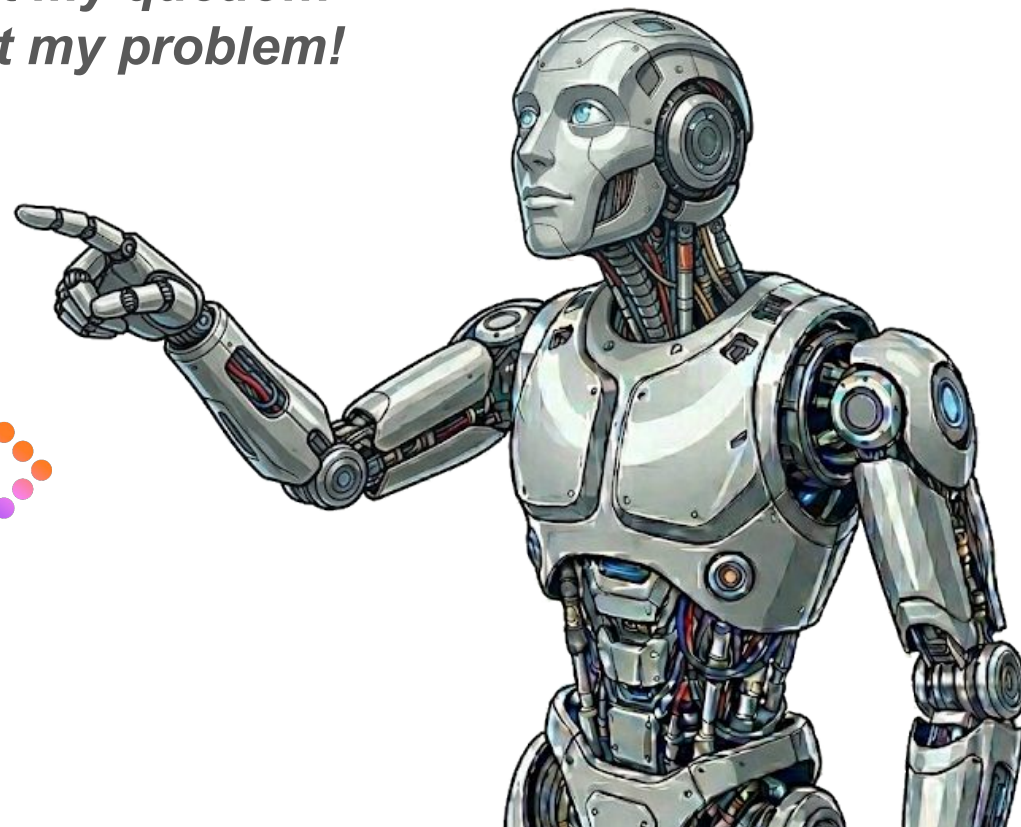
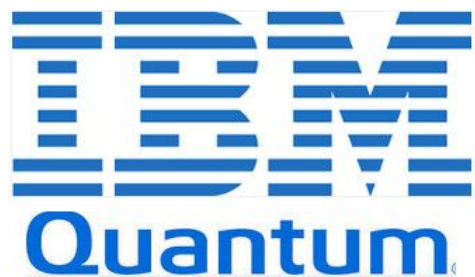


rigetti

IQM



*Not my queue...
not my problem!*

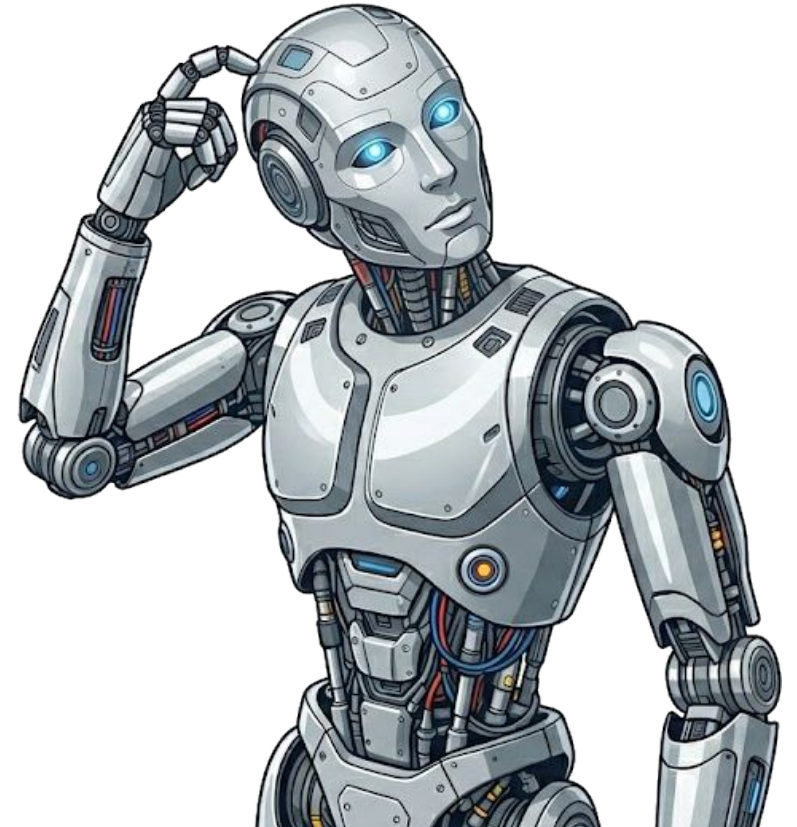
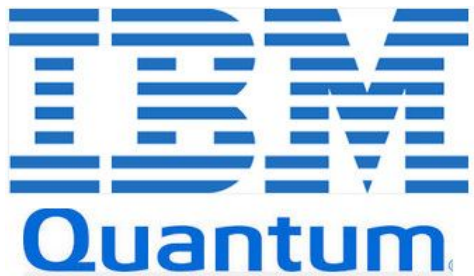




rigetti

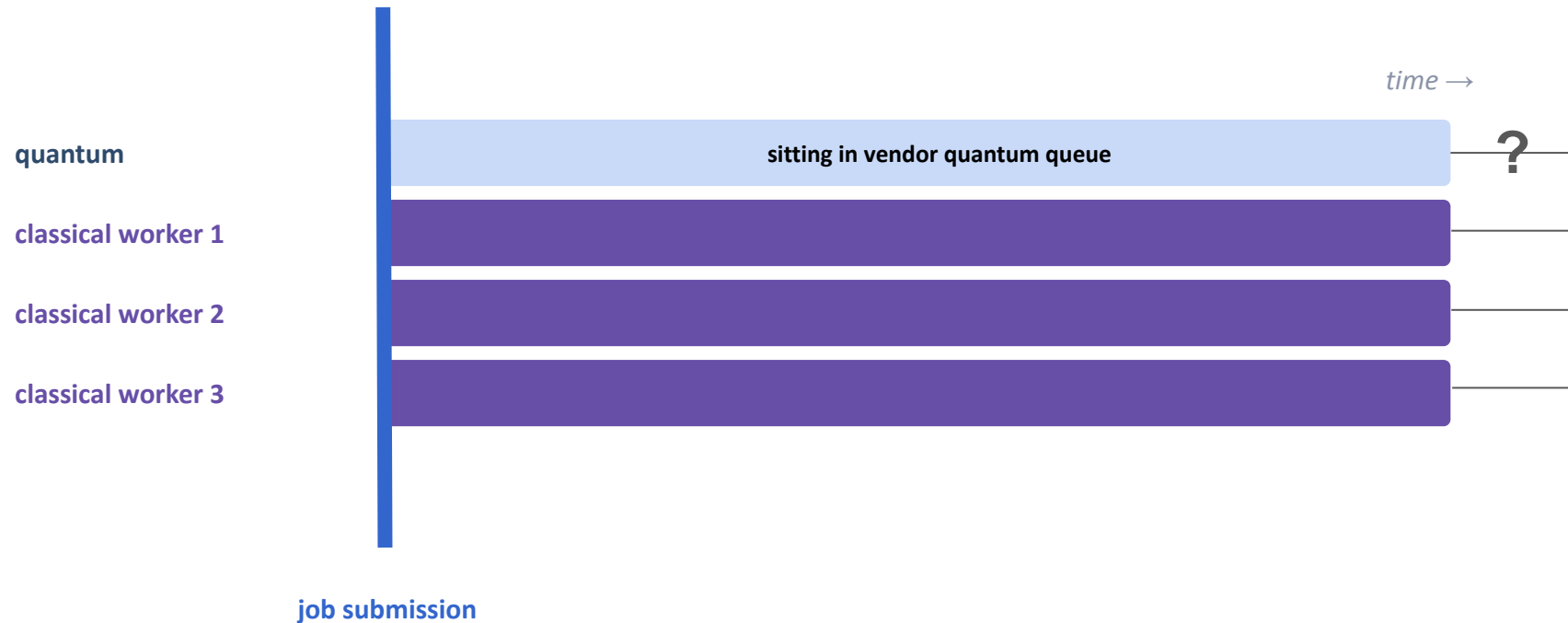
*Hey, can we synchronize
the classical and quantum
resources?*

IQM



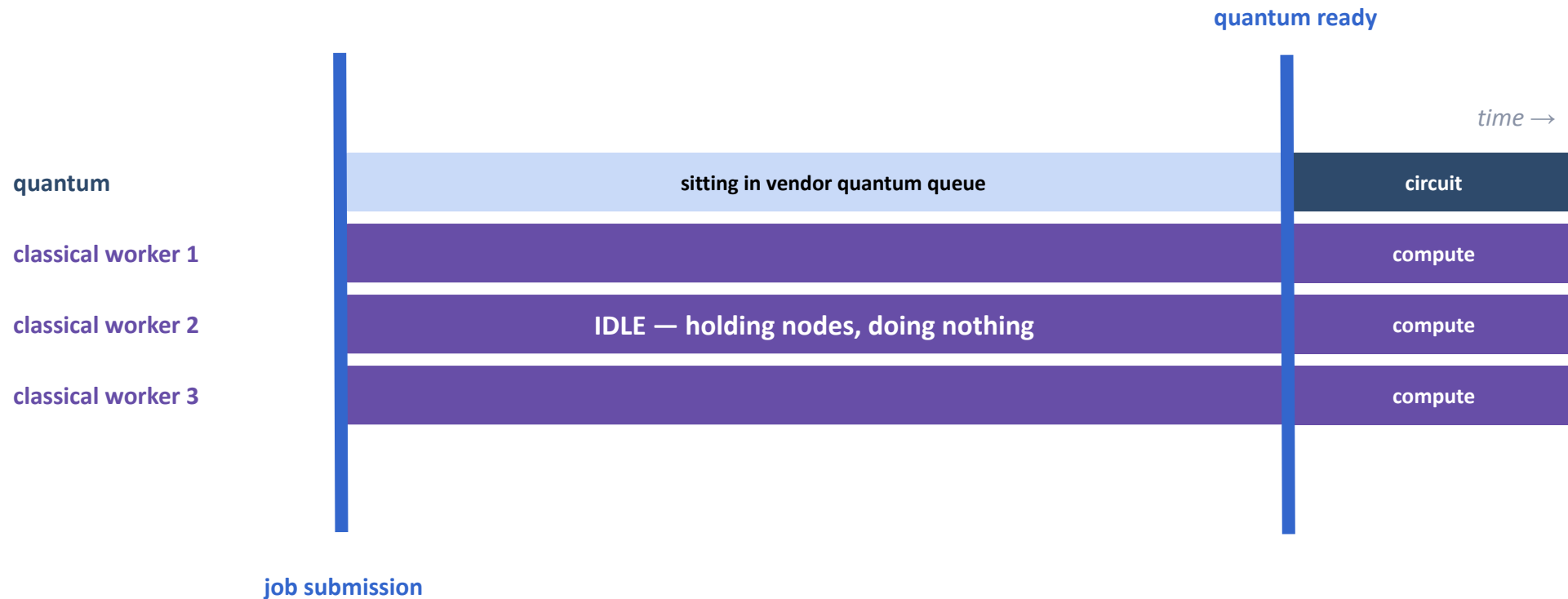
Two queues, only one of them ours

A hybrid quantum job is a classical job with a dependency that lives in "someone else's queue"



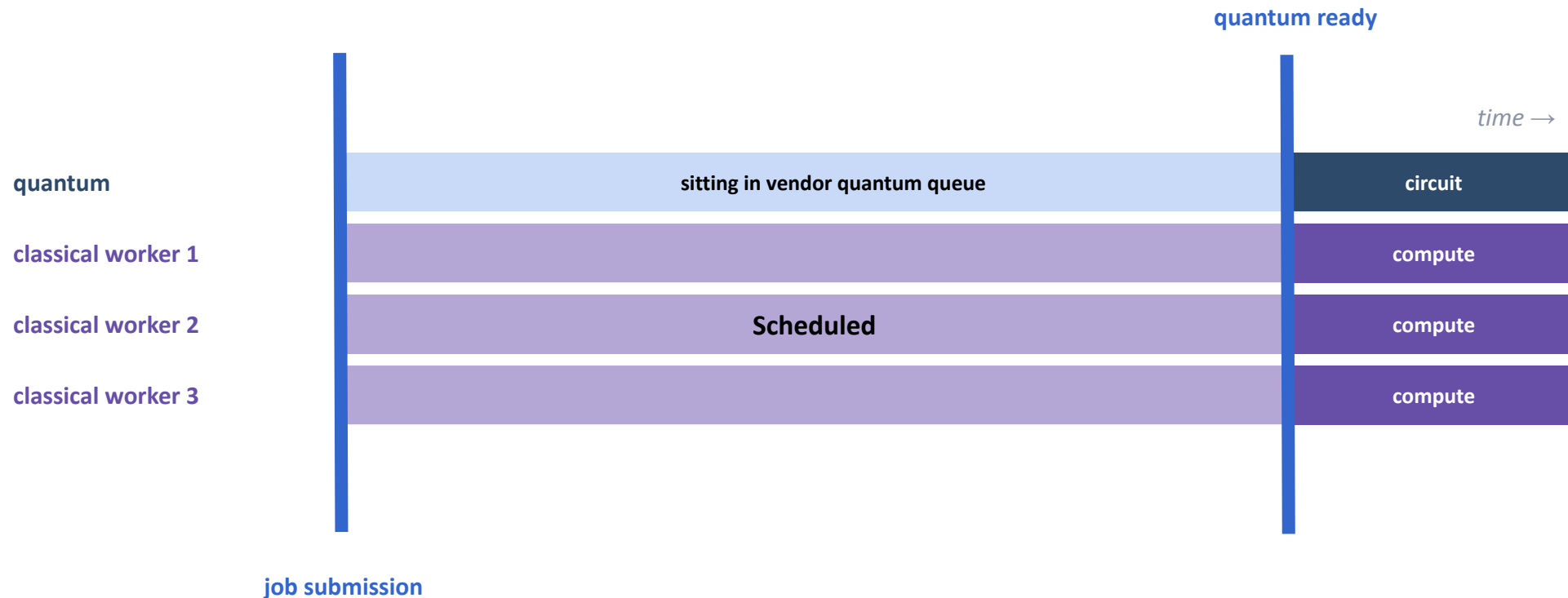
Two queues, only one of them ours

A hybrid quantum job is a classical job with a dependency that lives in "someone else's queue"



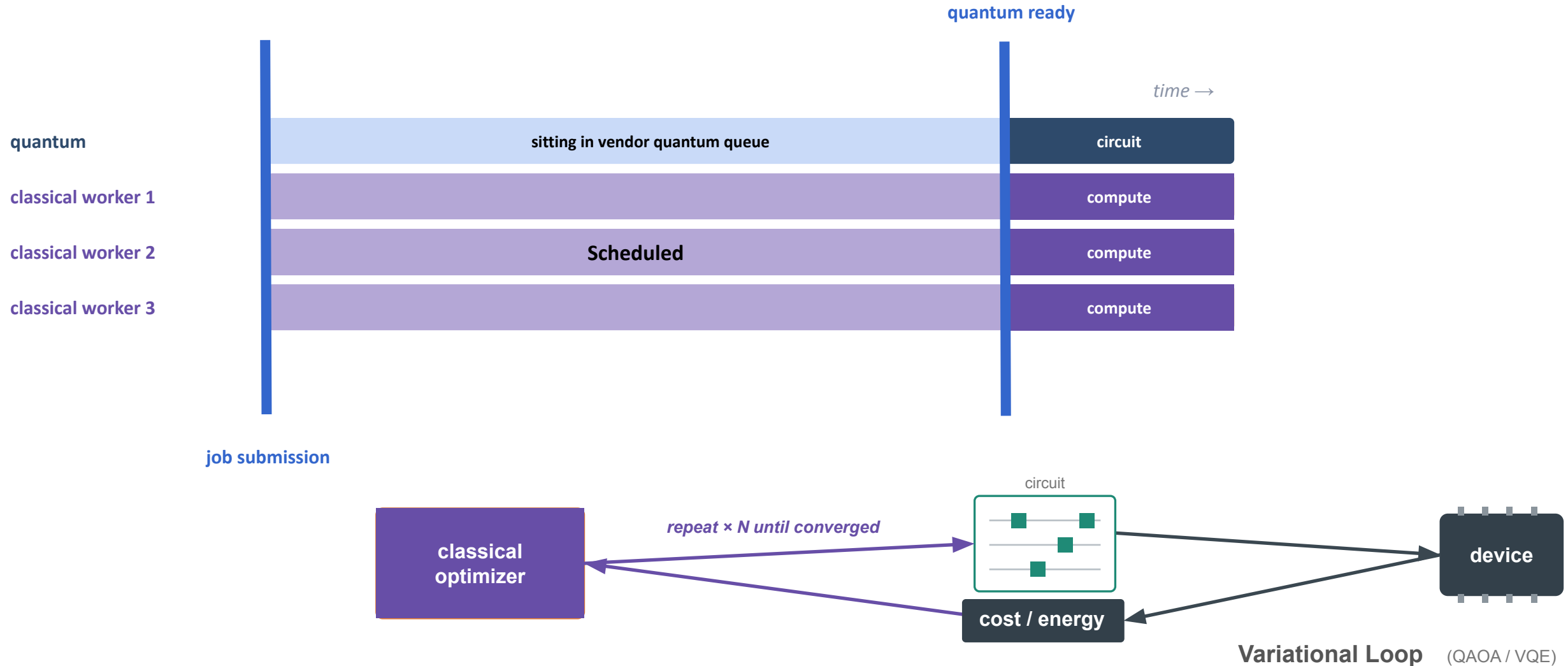
Intelligent co-scheduling means resources do not idle

We maximally start the classical work only when quantum is ready



Intelligent co-scheduling means resources do not idle

When work starts, repeated tasks have priority



FLUX FRAMEWORK FLUXION

Quantum Workload Manager Designs

Different strategies to handle the two-queue problem

Scheduling vs. Execution

Where does the integration live?

- **Scheduling integration:** The design models quantum as a schedulable entity, and given two different queues, manages co-scheduling to minimize idle time of waiting resources.
- **Execution integration:** The design is predominantly wrappers to handle environment, execution, and interaction with quantum resources.

Goals for Quantum Application Scheduling and Execution

How do designs intersect with scheduling?

We need co-scheduling designs that:

Goals for Quantum Application Scheduling and Execution

How do designs intersect with scheduling?

We need co-scheduling designs that:

- Eliminate or reduce the need for classical resources to wait for quantum resources (**the two queue problem**)

Goals for Quantum Application Scheduling and Execution

How do designs intersect with scheduling?

We need co-scheduling designs that:

- Eliminate or reduce the need for classical resources to wait for quantum resources (**the two queue problem**)
- Securely handle credentials to not give cross-exposure of application state or outputs between users.

Goals for Quantum Application Scheduling and Execution

How do designs intersect with scheduling?

We need co-scheduling designs that:

- Eliminate or reduce the need for classical resources to wait for quantum resources (**the two queue problem**)
- Securely handle credentials to not give cross-exposure of application state or outputs between users.
- Clearly define the separation between execution and scheduling.

Goals for Quantum Application Scheduling and Execution

How do designs intersect with scheduling?

We need co-scheduling designs that:

- Eliminate or reduce the need for classical resources to wait for quantum resources (**the two queue problem**)
- Securely handle credentials to not give cross-exposure of application state or outputs between users.
- Clearly define the separation between execution and scheduling.
- Are flexible for deployment with different institution setups and infrastructure stacks.

Workload Manager Design Buckets

How do designs intersect with scheduling?

Co-scheduling

Best effort to **schedule** and **run** quantum/classical together

Execution

Application, workflow tool, or workload manager component submits work **without regard for co-scheduling**.

Co-scheduling and Execution

Workload manager handles submission, monitoring, and co-scheduling.

Workload Manager Design Buckets

How do designs intersect with scheduling?

Co-scheduling

Best effort to **schedule** and **run** quantum/classical together

Execution

Application, workflow tool, or workload manager component submits work **without regard for co-scheduling**.

Co-scheduling and Execution

Workload manager handles submission, monitoring, and co-scheduling.

INTRODUCTION

Flux Framework

What makes Flux special?

What is Flux Framework?

Flux Framework "Flux" is a system or user-level **workload manager** developed by Lawrence Livermore National Laboratory that combines hierarchical **job management** with graph-based **scheduling**.

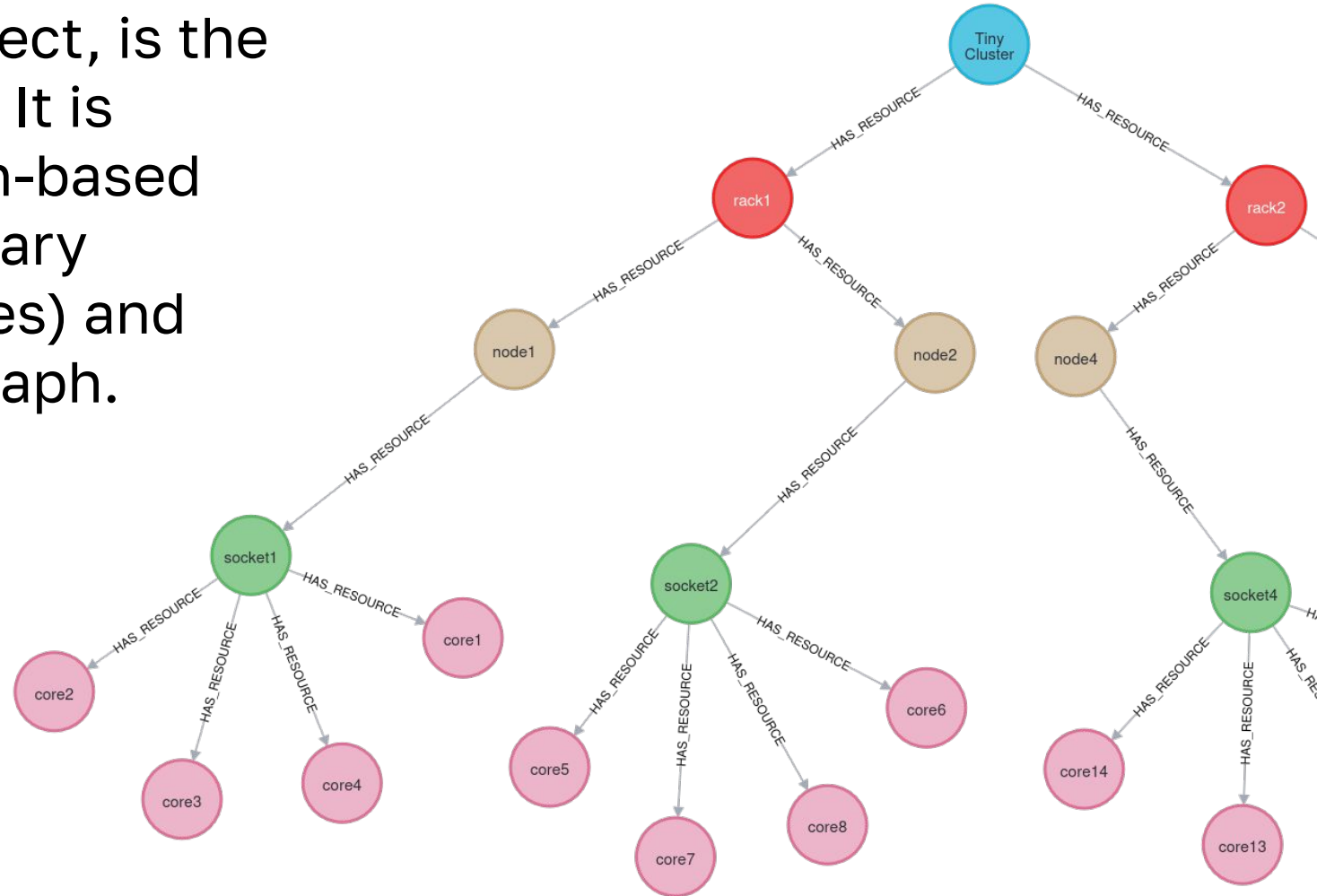
Did you know!

Flux recently joined the High Performance Software Foundation, and has 25+ contributors, including principal engineers behind Slurm, which also originated at LLNL.



What is Fluxion?

Fluxion, or the flux-sched project, is the **scheduler component** of Flux. It is uniquely responsible for graph-based scheduling, allowing for arbitrary definition of resources (vertices) and relationships (edges) in the graph.



Flux enables converged computing by design

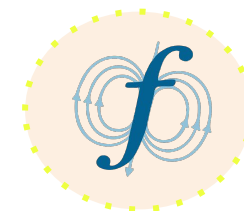
flux-core

flux-sched

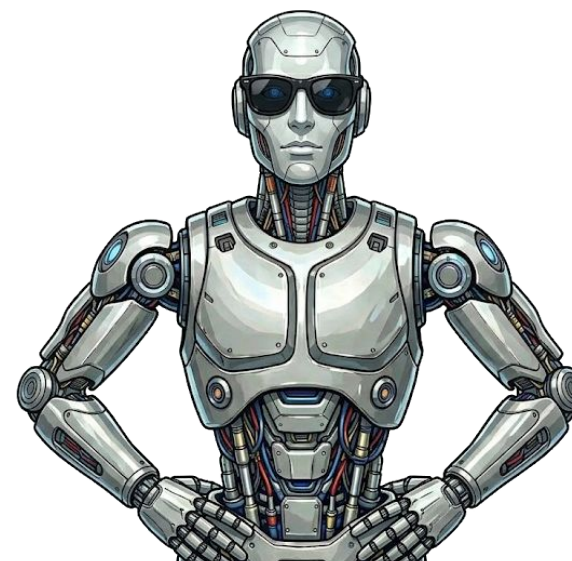
flux-security

flux-accounting

flux-pmix



Flux is *modular* by design.



Flux enables converged computing by design

flux-core

flux-sched

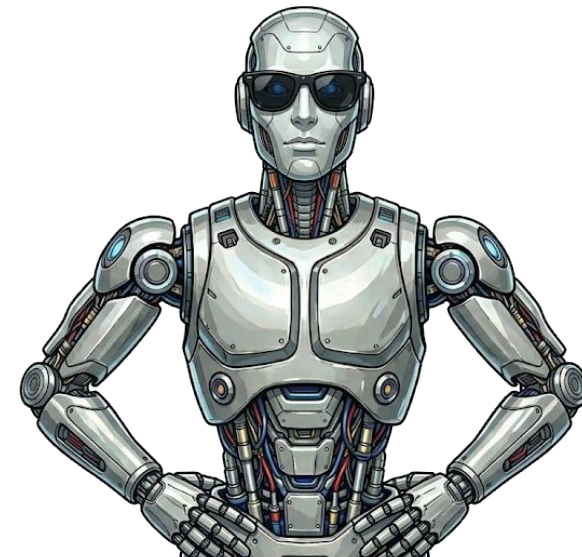
flux-security

flux-accounting

flux-pmix

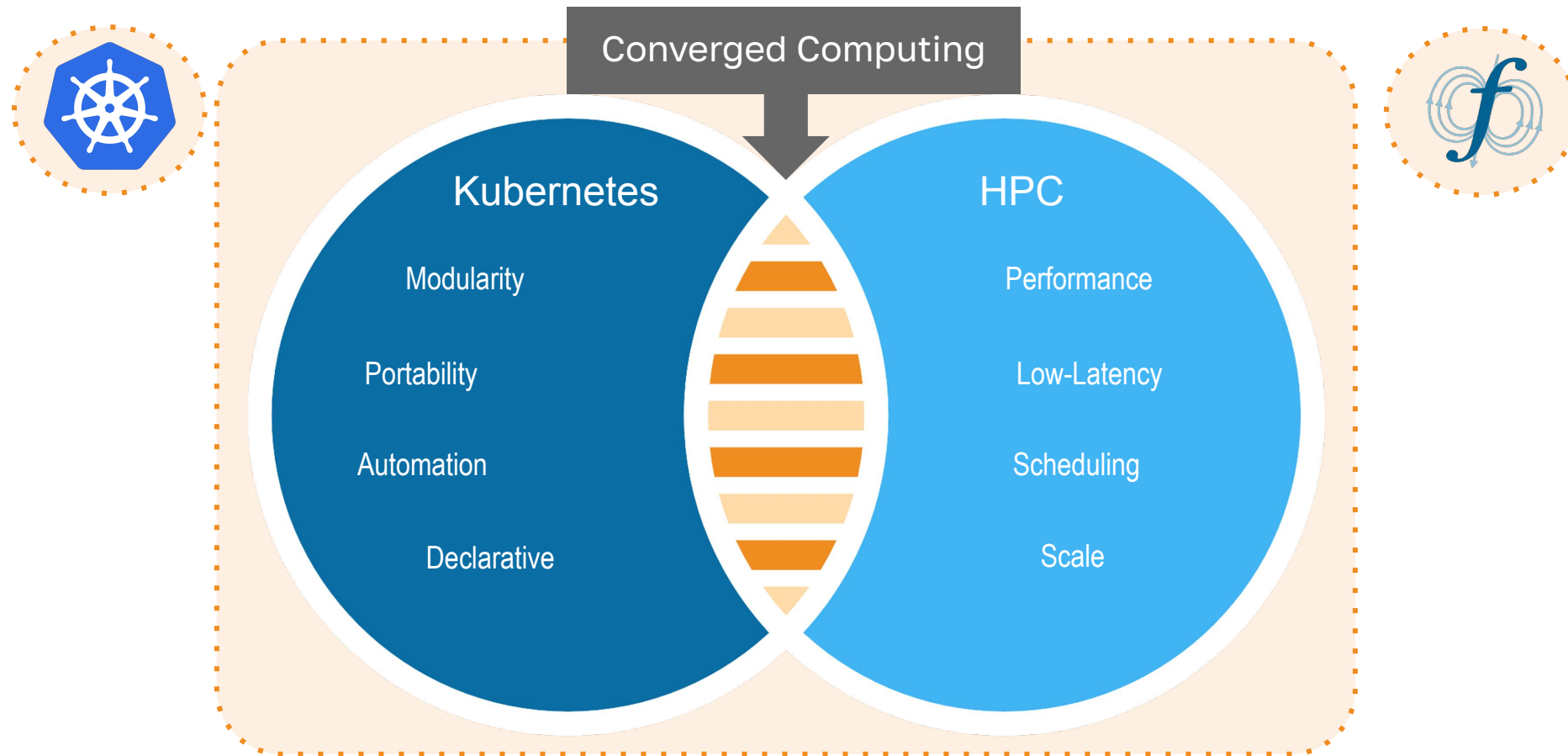


Flux is *modular* by design.



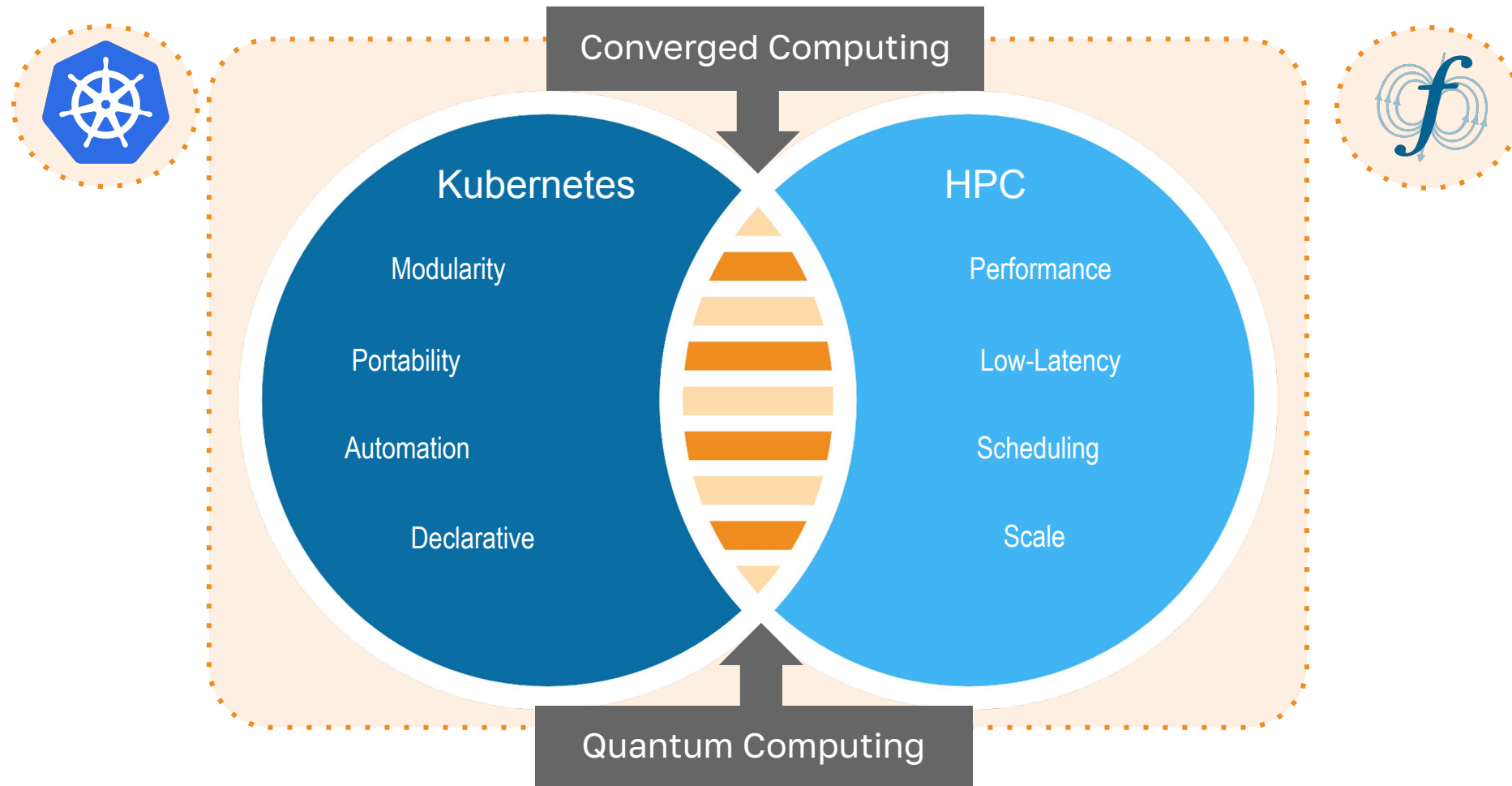
The best of both worlds between cloud-native and HPC.

Flux with Kubernetes meets the needs of agentic, combined AI/ML and simulation-based workloads



The best of both worlds between cloud-native and HPC.

Flux with Kubernetes meets the needs of agentic, combined AI/ML and simulation-based workloads



Flux enables converged computing by design

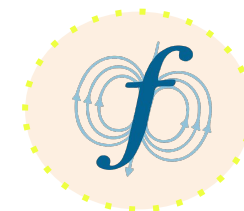
flux-core

flux-sched

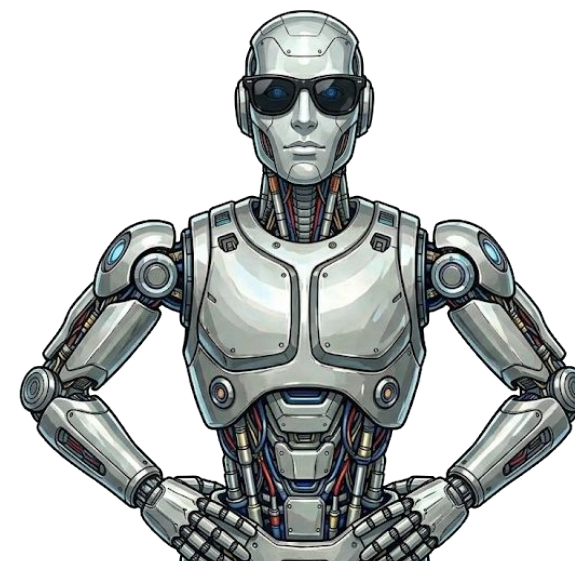
flux-security

flux-accounting

flux-pmix



Flux is *modular* by design.



Design 1: Kubernetes with Fluxion – "*fluence*"

*When we don't have complete control or transparency,
guaranteed and **predictable** is a good start.*

Fluence: One Step Closer to Converged Computing

Achieving Scalability with Cloud-Native HPC

- **Scale:** Updated design of MPI Operator to eliminate race conditions and scale MPI to over 3,000 ranks
- **Future:** Model for how to guide MPI-dependent applications from HPC to run declaratively in Kubernetes
- **Scheduling:** Gang scheduling done with Fluxion (flux-sched)
- **Performance evaluation:** Resolved "clogging" and waiting time of gangs to be scheduled inefficiently

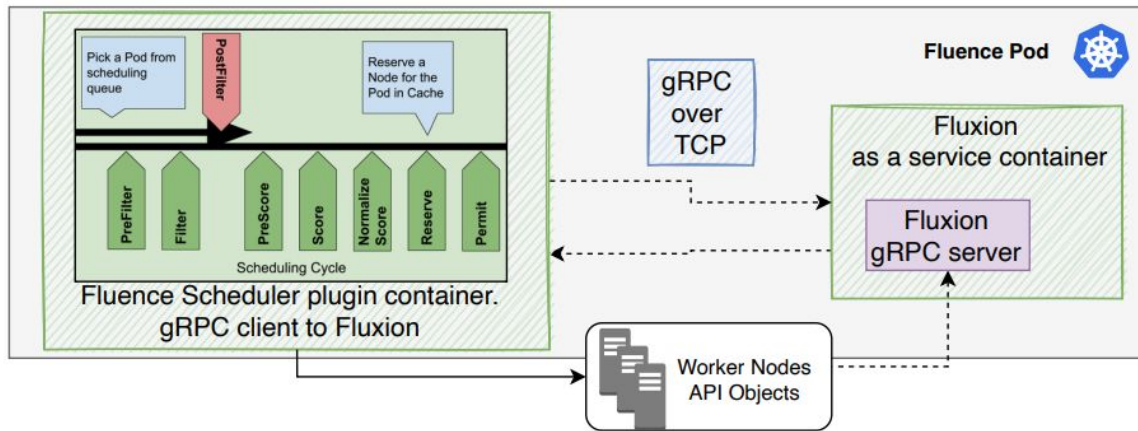


Fig. 2: Implementation of Fluence; K8s Extension Points are indicated with green labels.

Fluence: One Step Closer to Converged Computing

Achieving Scalability with Cloud-Native HPC

- **Scale:** Updated design of MPI Operator to eliminate race conditions and scale MPI to over 3,000 ranks
- **Future:** Model for how to guide MPI-dependent applications from HPC to run declaratively in Kubernetes
- **Scheduling:** Gang scheduling done with Fluxion (flux-sched)
- **Performance evaluation:** Resolved "clogging" and waiting time of gangs to be scheduled inefficiently

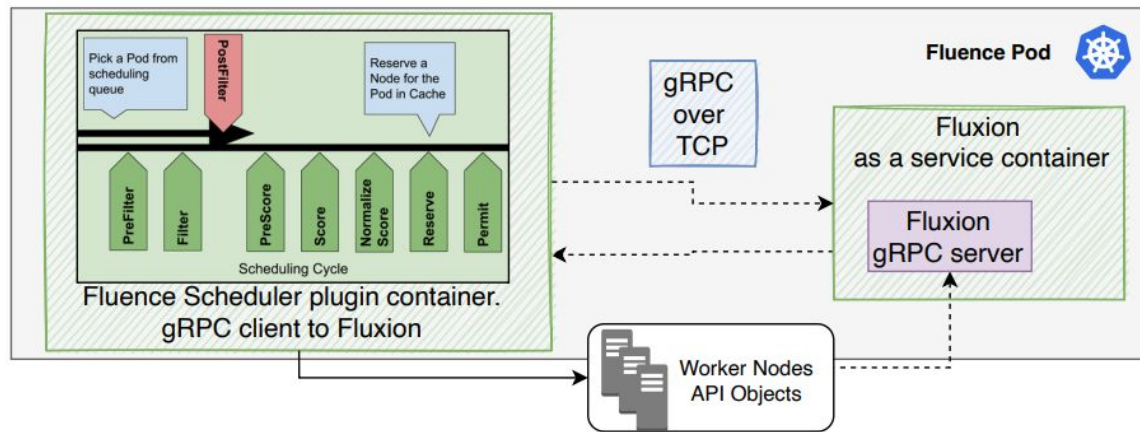
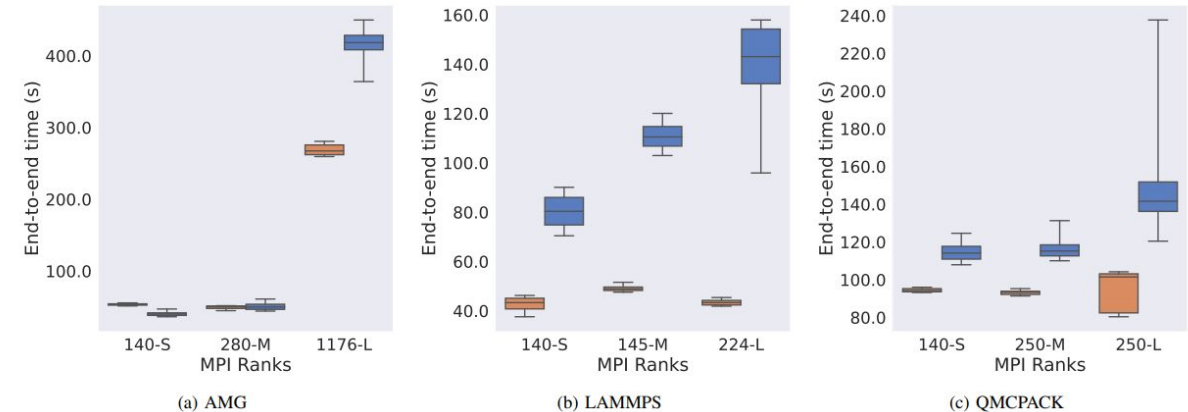
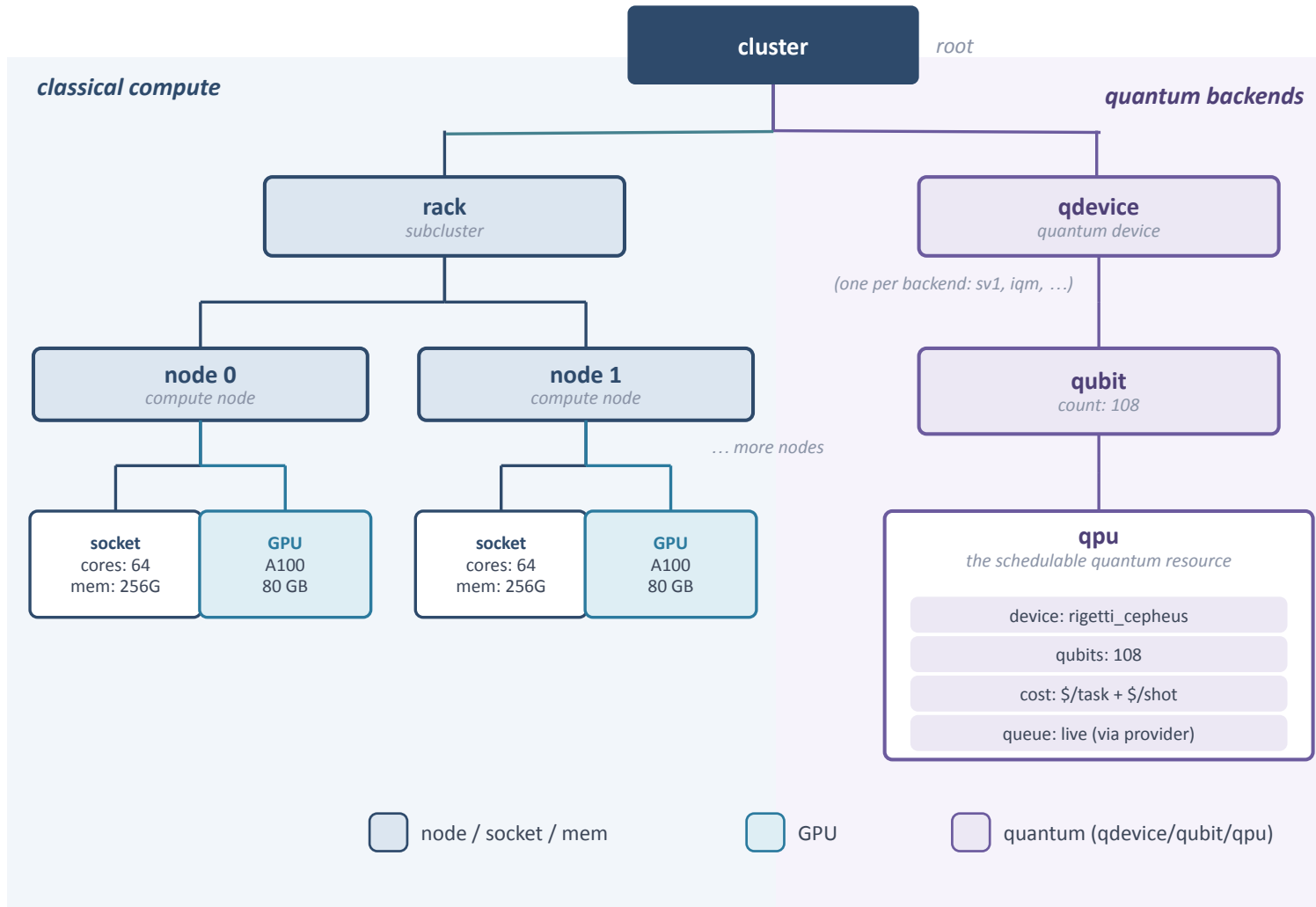


Fig. 2: Implementation of Fluence; K8s Extension Points are indicated with green labels.



Quantum is modeled as a first class resource

A quantum device sits at the same level as a rack; a job request is a traversal over the graph



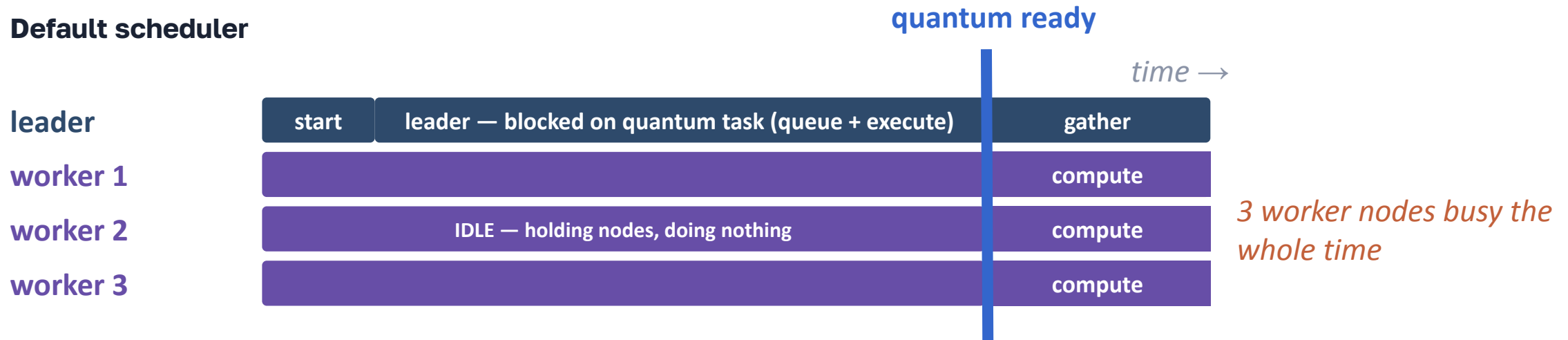
The idea

Fluxion can query a single resource graph looking for resources of arbitrary, custom types like classical alongside quantum. We can set constraints to select on vendor, device, or class.

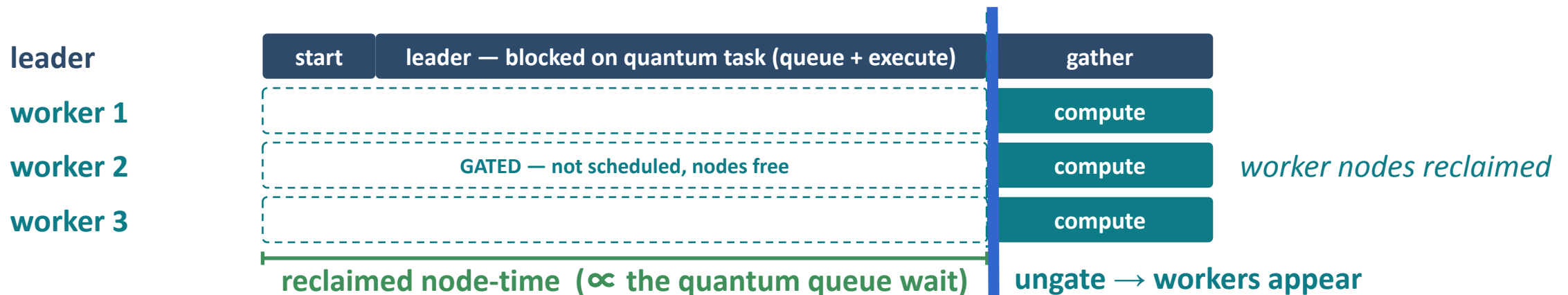
Fluence: using gating to address the two-queue problem

"I want to co-schedule quantum resources (qpus) for my classical job..."

Default scheduler



Fluence (gated)



Fluence: using gating to address the two-queue problem

"I want to co-schedule quantum resources (qpus) for my classical job..."

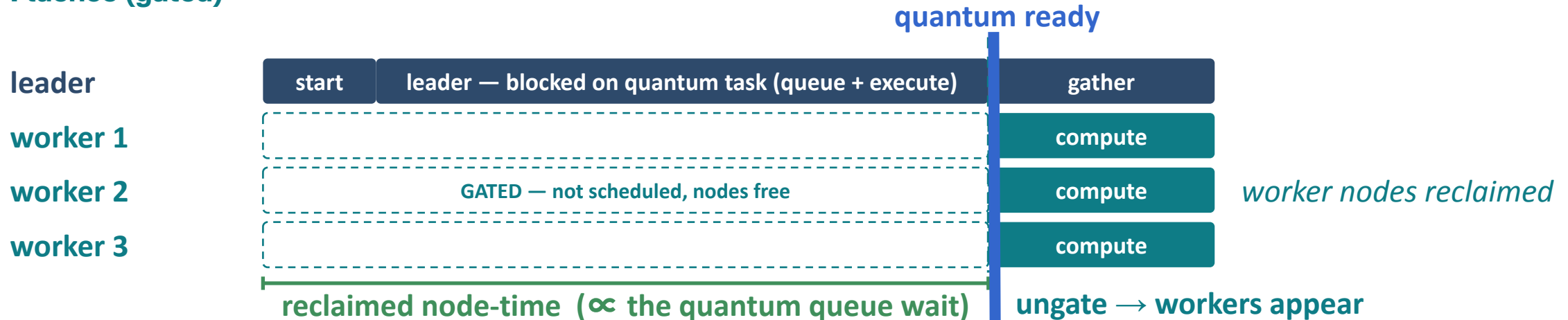
What the user writes

```
# quantum-pod.yaml
schedulerName: fluence
resources:
  limits:
    fluxion.flux-framework.org/qpu: "1"
```

No group. No hard-coded backend — the webhook + PreBind supply it.

```
$ kubectl logs sampler -f
submitting sampler job to
  ibm_marrakesh
done: 2070 bytes from
  ibm_marrakesh
```

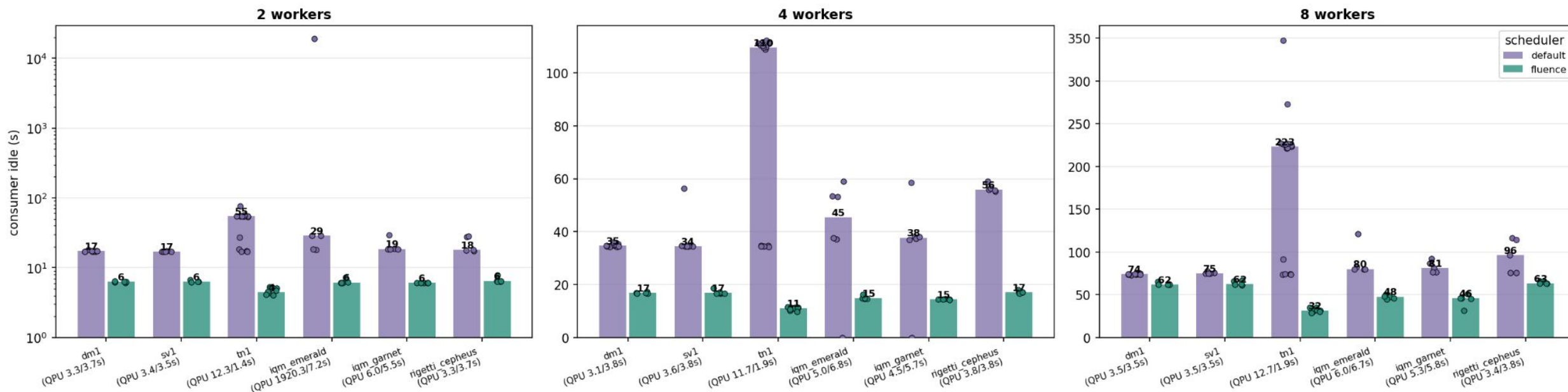
Fluence (gated)



A gating strategy eliminates worker idle time

Gating consumers until the quantum task is ready turns the two-queue wait into near-zero idle

Consumer idle (all backends): fluence vs default

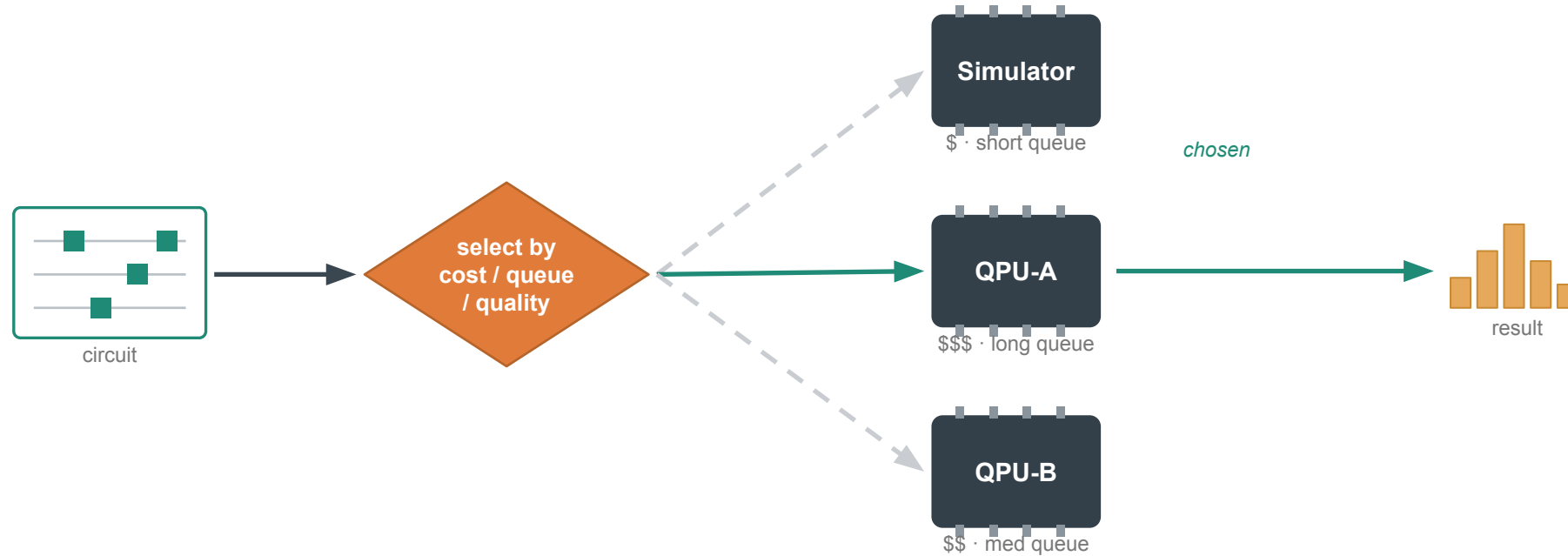


19,160

Node-seconds a default gang idled on a single 2.66-hour device queue. Fluence gates the consumers — cutting idle roughly 5× on short queues and by orders of magnitude on long ones.

Pattern 5: Backend selection (ensemble)

Pick the best device for the job (cheapest, shortest queue, etc.)



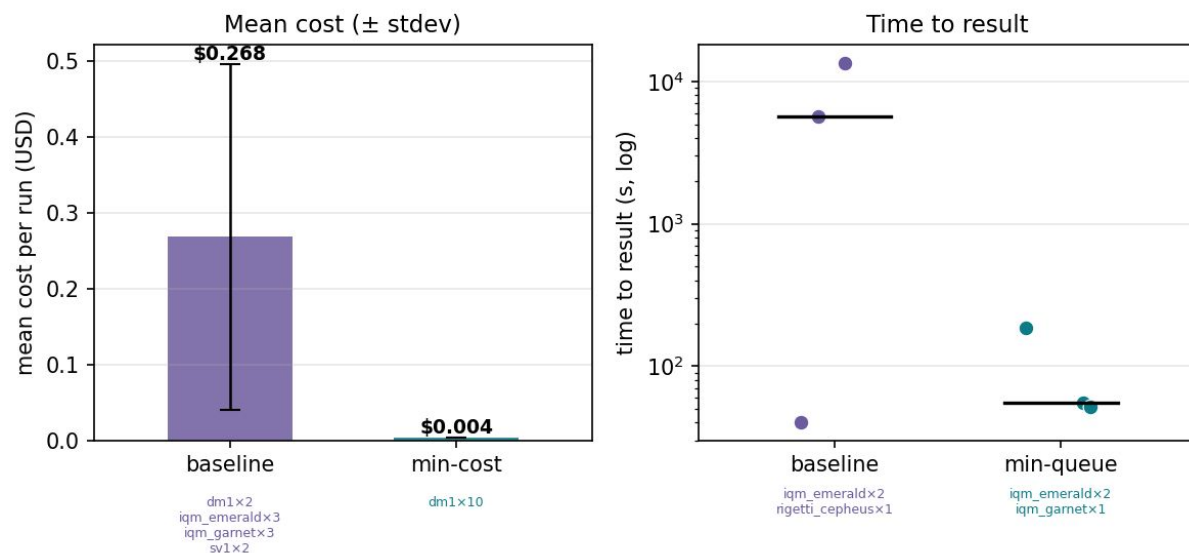
Shape: one job with several candidate devices. You can either choose one by attributes or run multiple and combine.

Cost- and queue-aware backend selection

A kubectl plugin queries queue depth and price locally, then pins the best device for the job



Cost- and queue-aware selection



What the policy buys you

- Min-cost pinned dm1 on every run — mean cost \$0.268 → \$0.004.
- Roughly 72× cheaper per run, and the cost variance is eliminated.
- Min-queue selection cut time-to-result from hours to under a minute for comparable backends.

Design 2: Flux Framework and Reservation

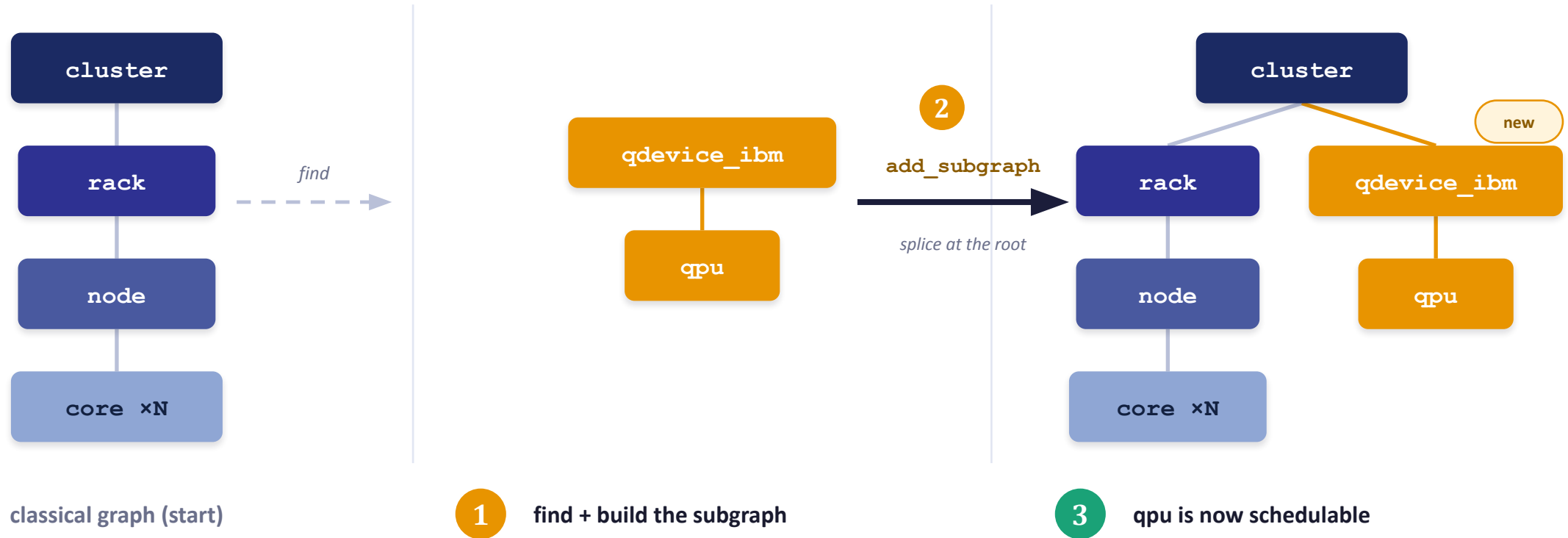
*When we don't have complete control or transparency,
guaranteed and **predictable** is a good start.*

Design 2: Flux Framework and Hold

*When we don't have complete control or transparency,
guaranteed and **predictable** is a good start.*

Inject custom resources in the Fluxion graph

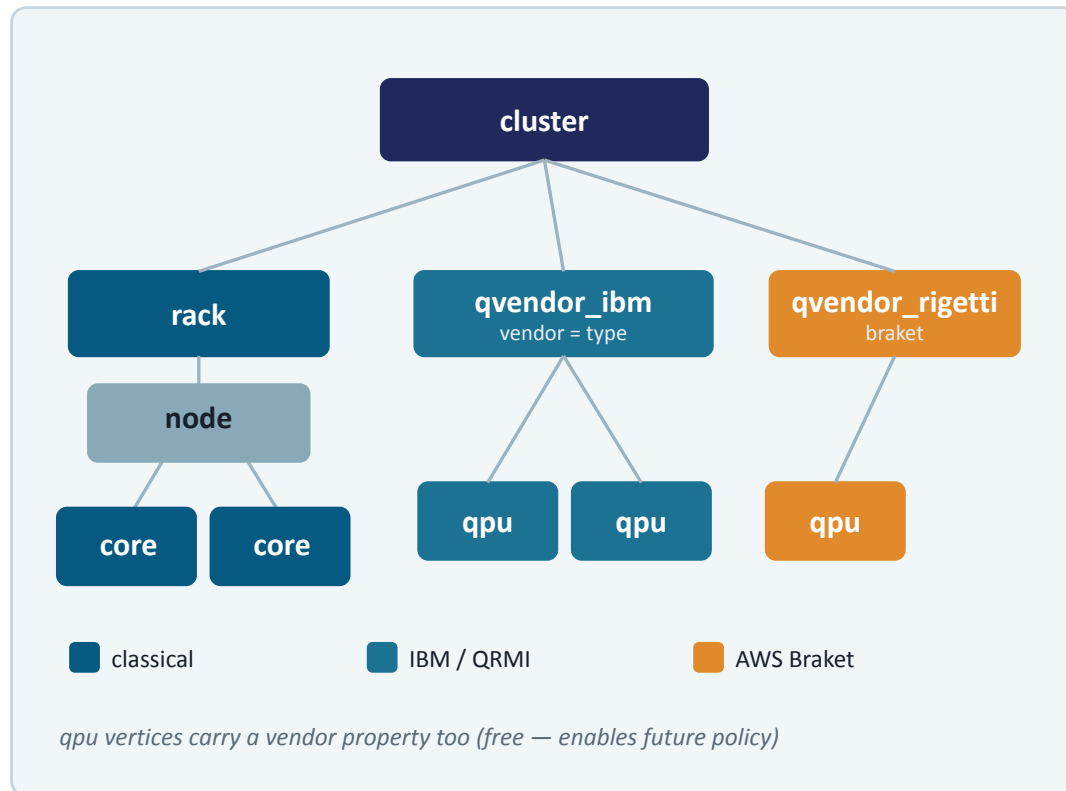
Splice custom resources (quantum) into system or instance-level Fluxion graph.



We use the **find** rpc (1) to get live resources, and then the **add_subgraph** rpc (2) to add our custom resources (like quantum) into it, either on the level of a system or user instance. The result (3) is that we have a schedulable (traversable) resource.

Model vendors in the resource graph

To optimize the query, model vendors as types directly in the Flux scheduler graph



qpu must be requested exclusive

A non-exclusive leaf device is required for the match but silently dropped from R — fluxion's upd_plan adds the span only under if(excl).

load with match-format=rv1

rv1_nosched (the default) strips the scheduling/JGF section where custom types live. **rv1** keeps it, so the qpu appears in the job's R.

Reserve classical first, hold classical until its ready

How will we co-schedule with Flux given the two-queue problem?

External quantum

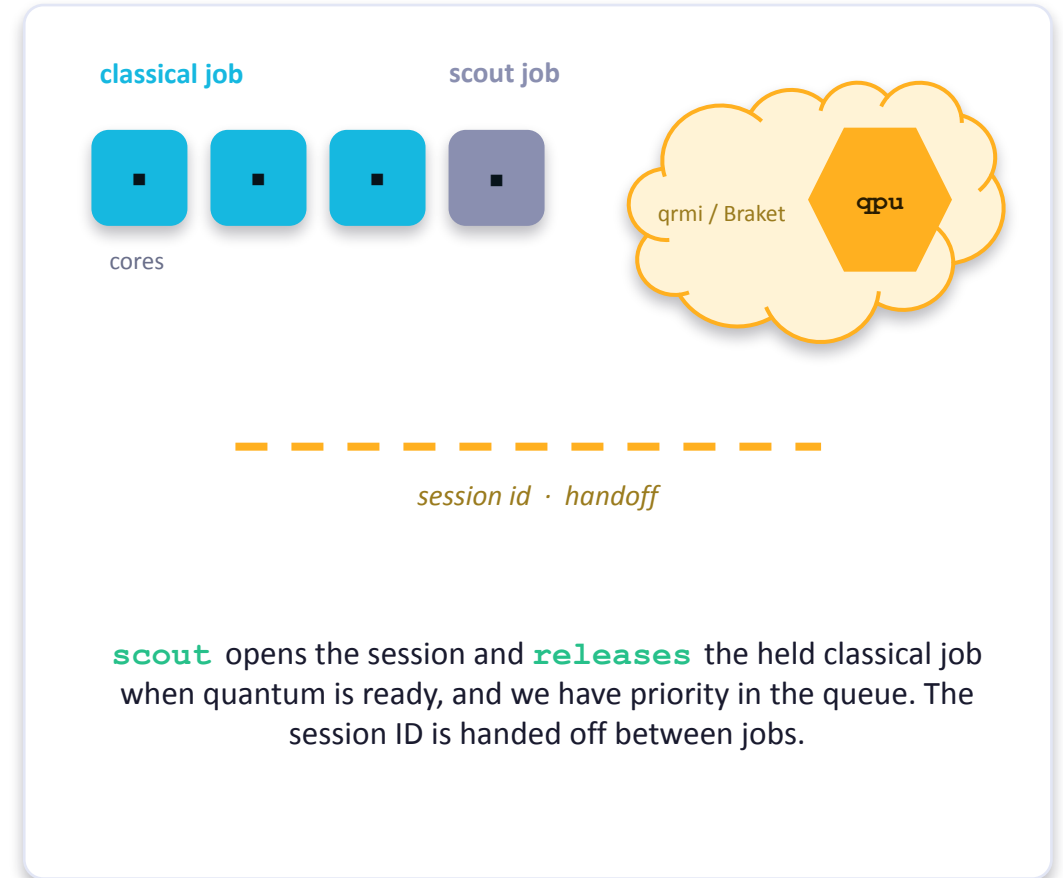
We do not own the hardware for QRMI vendors or AWS Braket. We need to allow the user to select devices or properties for metered per-use quota.

Never idle resources

We do not request a vendor session (**scout**) until classical reservation is satisfiable by the fluxion scheduler. The **scout** will use an RPC to release classical resources.

One user action

A single flux submit is extended via a client plugin to take a single request and dispatch both the scout job and classical reservation. All credentials remain in user space.



Flux client plugin to modify "flux submit"

An easy means to add a `--quantum-vendor` (or other attribute) request for a job

```
$ flux submit --quantum-vendor ibm --quantum-ibm-backend ibm_brisbane -- my-vqe --mol H2
```

CLI plugin
`modify_jobspec()`



classical HOLD

The user application (my-vqe), **wrapped** to wait for the session via `hold=1` (reserved). Submitted by a Flux client plugin. The scout and classical are a pair that move through scheduling together. If the classical are unsatisfiable, both are aborted.



scout RUN

this submit is **rewritten** into the scout: a small foothold + the exclusive qpu, running scout.py.
flux submits THIS — so the returned id is the scout's.

Flux client plugin to modify "flux submit"

An easy means to add a `--quantum-vendor` (or other attribute) request for a job

```
$ flux submit --quantum-vendor ibm --quantum-ibm-backend ibm_brisbane -- my-vqe --mol H2
```

CLI plugin
`modify_jobspec()`



classical HOLD

```
resources:
- type: slot
  with:
    - type: node
      count: 4
      with: [{type: core, count: 8}]
attributes: { system: { hold: 1 } }
tasks: [ quantum-wrap -- ./my-vqe -mol H2 ]
```



scout RUN

```
resources:
- type: slot
  with:
    - {type: core, count: 1}
    - type: qvendor_ibm
      with: [{type: qpu, count: 1}]
tasks: [ quantum-scout --job <main> ]
```

Flux client plugin to modify "flux submit"

An easy means to add a `--quantum-vendor` (or other attribute) request for a job

```
flux submit -N 4 -n 16 --quantum-vendor ibm ./app
```

opens session → unholds ②

Flux Framework

scout job

worker node 1

worker node 2

worker node 3

classical job

leader — blocked on quantum task (queue + execute)

HELD (reserved)

Flux client plugin to modify "flux submit"

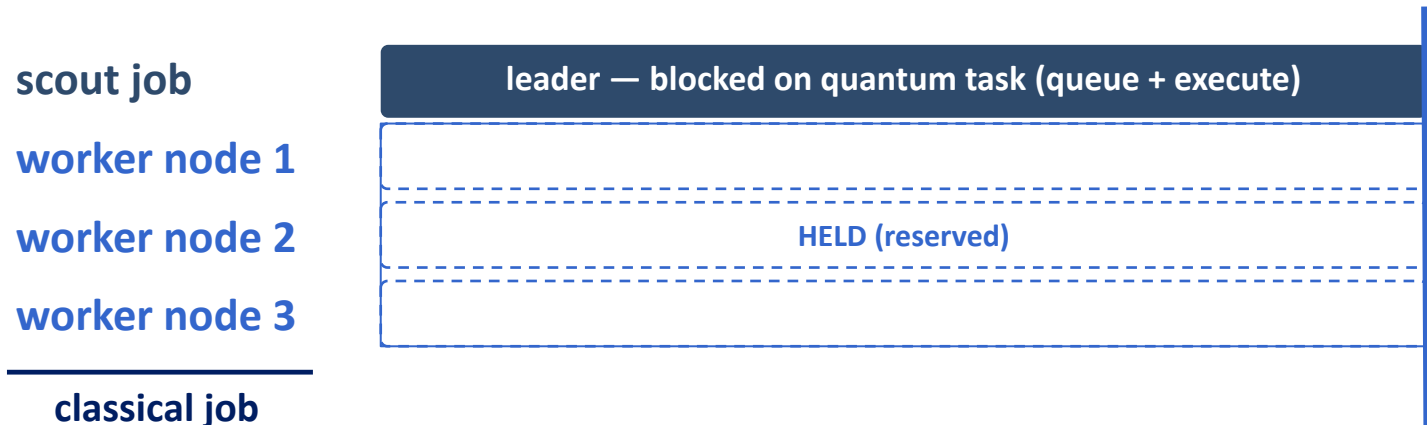
An easy means to add a `--quantum-vendor` (or other attribute) request for a job

```
flux submit -N 4 -n 16 --quantum-vendor ibm ./app
```

opens session → unholds ②

Flux Framework

quantum ready, priority queue
→ `sched-fluxion-qmanager.release` (rpc)



Flux client plugin to modify "flux submit"

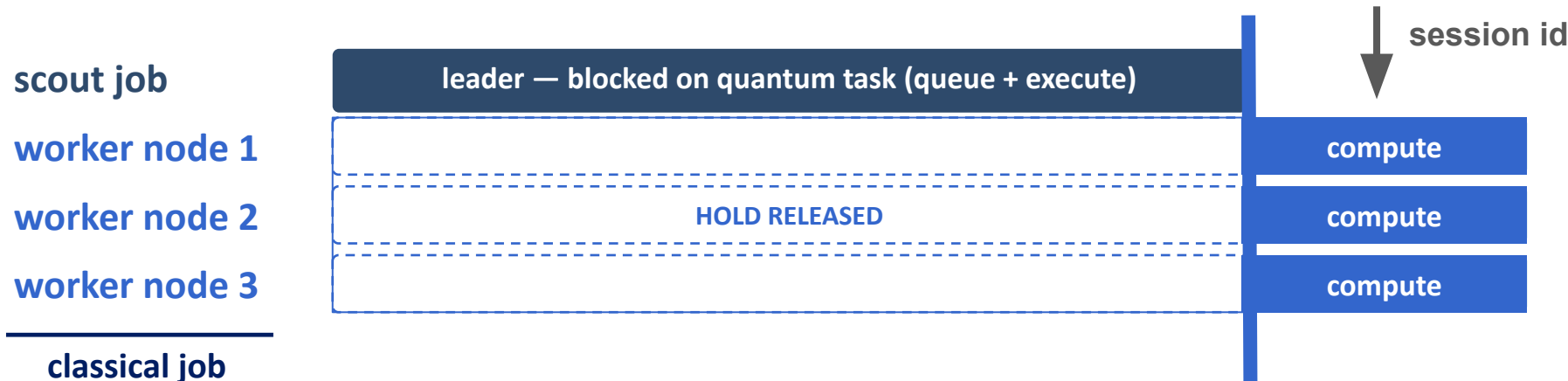
An easy means to add a `--quantum-vendor` (or other attribute) request for a job

```
flux submit -N 4 -n 16 --quantum-vendor ibm ./app
```

opens session → unholds ②

Flux Framework

quantum ready, priority queue
→ `sched-fluxion-qmanager.release` (rpc)



How do we include vendor-specific logic?



The client scout launchers are backend-specific, a standard interface to submit to Flux

mock

implemented

```
--quantum-mock-session  
--quantum-mock-latency
```

qrmi (ibm)

```
--quantum-ibm-backend  
--quantum-ibm-shots  
--quantum-ibm-instance
```

braket

```
--quantum-braket-device  
--quantum-braket-region  
--quantum-braket-shots
```

🚧 Under development! 🚧

This interface will likely undergo changes. It's current purpose is to support experiments and learning.

Quantum-Classical coscheduling in Flux

flux-sched supports generic custom resources, and flux-quantum uses interfaces



flux-sched

the MECHANISM — quantum-agnostic

- hold / release RPCs (qmanager)
- add_subgraph / find (resource)
- reserved-first held jobs
- no vendor or resource specific knowledge

depends on



flux-quantum

the SHAPE + POLICY — vendor-aware

- Defines device shapes for the graph
- CLI plugin — the jobspec split + gate
- Vendor selection + backends
- Scout — session open + rendezvous

Design 3: Workflow Tools

Place responsibility for coordination and execution on a workflow tool.

```

Snakefile

# This is a Snakefile - this is akin to the "all" the final result.
rule all:
    input:
        "results/qaoa_hardware_costs.csv"

# Step 1: Create a secure QRMI authentication context session
# Simple way to create a token that could be grabbed by multiple (parallel) tasks
rule qrmi_create_session:
    output:
        token_env = "security/qrmi_session.env"
    log:
        "logs/qrmi_init.log"
    wrapper:
        "github://quiskit/qrmi-wrappers/init_session/@v1.0.0"

# Step 2: Run a well-known quantum workflow (QAOA Max-Cut) using the token
rule run_qaoa_workflow:
    input:
        token_env = "security/qrmi_session.env",
        graph_data = "data/problem_graph.json"
    output:
        results = "results/qaoa_optimization_metrics.json",
        cost_log = "results/qaoa_hardware_costs.csv"
    log:
        "logs/qaoa_execution.log"
    params:
        p_steps = 3,
        backend = "ibm_torino" # Supports QRMI ecosystems like IBM or Pasqal
    wrapper:
        "github://quiskit/qrmi-workflows/quantum_workflows/qaoa/@v1.0.0"

```

Snakemake wrappers for QRMI

<https://github.com/converged-computing/quantum-wrappers>

- Application- and vendor-specific logic for execution provisioned as reusable, customizable steps
- Best practices and expertise maintained in wrappers (rules)
- Assemble new workflows from component steps
- No issue with submission timing for serial tasks.

A workflow can orchestrate quantum, but cannot coschedule

Execution and monitoring is not reservation. We might still end up waiting on uncoordinated Qs

WHAT A WORKFLOW TOOL DOES

rule qrmi_create_session

output: qrmi_session.env

qrmi_session.env

rule run_qaoa_workflow

backend=ibm_torino · p_steps=3

qaoa_hardware_costs.csv

rule all

the final result

So it genuinely can:

- give each rule its own env, wrapper, params
- run steps in dependency order (a DAG)
- submit each step to a queue, then wait
- create a session, then run a workflow using it

Great for orchestration — decides WHEN each step runs.

A workflow can orchestrate quantum, but cannot coschedule

Execution and monitoring is not reservation. We might still end up waiting on uncoordinated Qs

WHAT A WORKFLOW TOOL DOES

```
rule qrmi_create_session
output: qrmi_session.env
```

qrmi_session.env

```
rule run_qaoa_workflow
backend=ibm_torino · p_steps=3
```

qaoa_hardware_costs.csv

```
rule all
the final result
```

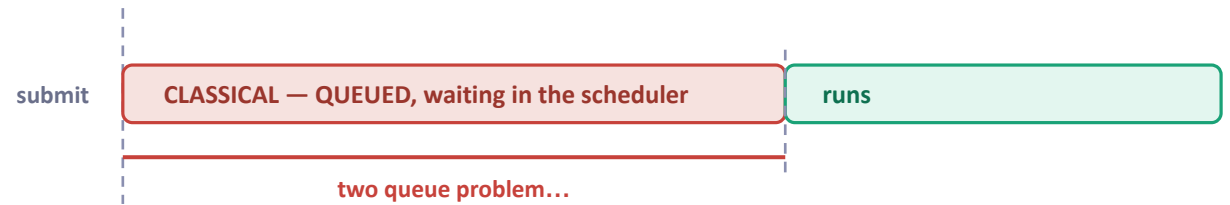
So it genuinely can:

- give each rule its own env, wrapper, params
- run steps in dependency order (a DAG)
- submit each step to a queue, then wait
- create a session, then run a workflow using it

Great for orchestration — decides WHEN each step runs.

ORCHESTRATION != COSCHEDULING

workflow tool — scout says “ready”, then you submit the classical



Serial submissions will work OK



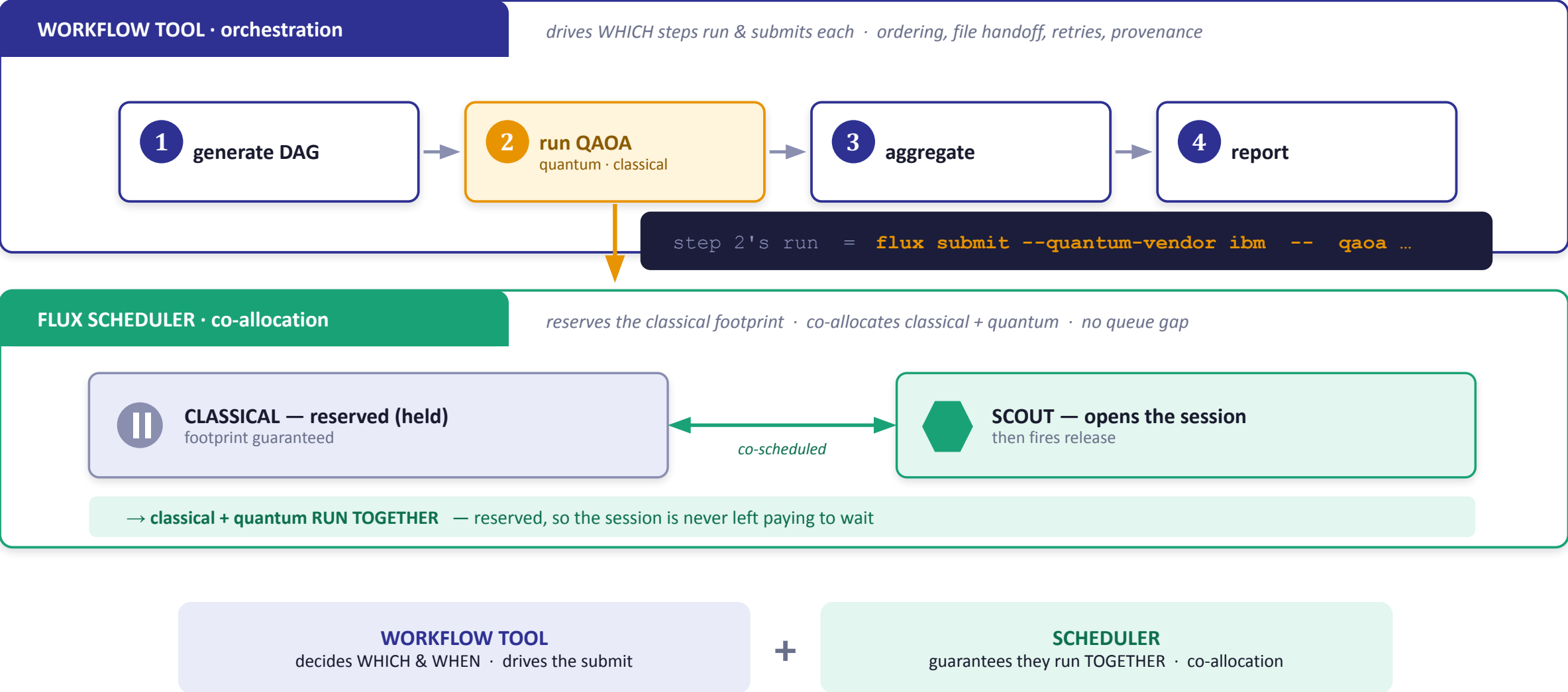
The scout trick can't rescue a workflow tool

Its next move is a job SUBMIT — which lands in a queue. Unless every step runs strictly serial on resources you already hold, the classical waits and the metered session waits with it.

A workflow tool decides WHEN steps run. Co-scheduling makes them run TOGETHER — by reserving the classical footprint first.

Workflow manager and workflow manager as a team ❤️

The workflow tool drives execution, and the scheduler guarantees



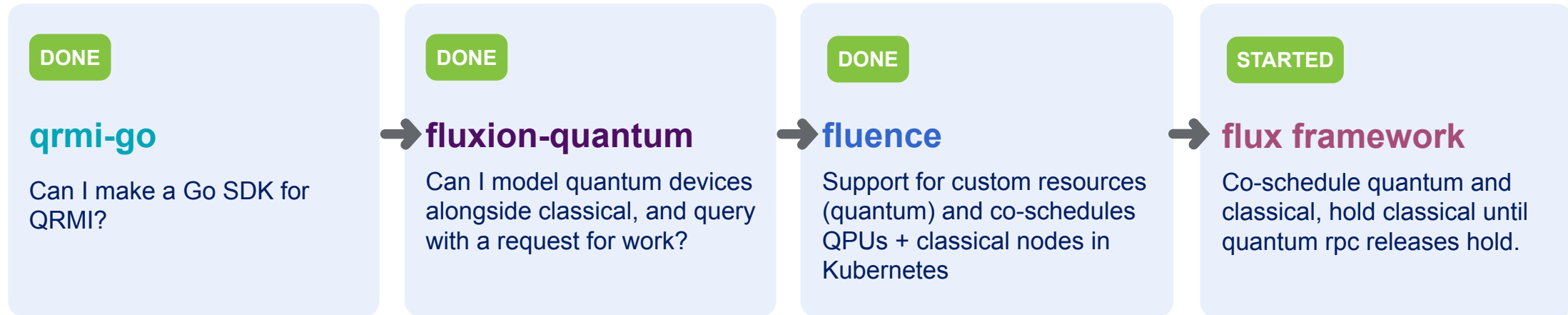
CONCLUSION

Summary

What have we learned so far?

From prototype to a working schedulers

How did we go from "I know nothing" to Flux Framework and Kubernetes prototypes?



-  github.com/converged-computing/qirmi-go
-  github.com/converged-computing/fluxion-quantum
-  github.com/converged-computing/fluence
-  github.com/vsoch/flux-sched



 Flux Framework and Quantum: Early Work

Talk and Demos on YouTube

Working on quantum scheduling is exciting



What did we cover today?

- Application design (needs) is the primary driver of scheduler designs
- The second queue problem (extended to the N-queue problem) requires coordination of queues
- We can model quantum devices as first class citizens in a resource graph
- Workflow tools can collaborate to optimally package steps, execute, and coschedule.

Feedback for vendors and Future Work



How can vendor and open source APIs be better help with coscheduling?

Feedback

- For QRMI, the *acquire* might return a session ID that is not a task submit, but "foot in the queue"
- Two means for client to know when quantum is ready:
 - Polling of the task ID (works today)
 - Event driven notifications (ideal)

Feedback for vendors and Future Work



How can vendor and open source APIs be better help with coscheduling?

Feedback

- For QRMI, the *acquire* might return a session ID that is not a task submit, but "foot in the queue"
- Two means for client to know when quantum is ready:
 - Polling of the task ID (works today)
 - Event driven notifications (ideal)

Future Work

- Experiments with Flux core and QRMI/Braket with Flux Framework
- Discuss other designs (e.g., single job that grows when quantum ready)
- Extending co-scheduling designs to local quantum devices
 - no two queue problem, we own and control the resources, a single queue

👁️ Looking forward

Quantum-awareness can be added to other schedulers with analogous design

By modeling a remote QPU as a first-class graph resource and gating classical workers on the two-queue wait, many schedulers can co-schedule quantum and classical work to reclaim node idle time, cut cost, and improve utilization.

One resource graph, from CPUs to QPUs.



Thank you!

sochat1@llnl.gov | milroy1@llnl.gov

